

# Python Programming Notes

Jupyter Notebook Export

## Python Programming Notes

*Converted from handwritten class notes*

### Number of Approaches to Develop the Program in Python

In Python programming, we have **2 types of approaches** to develop the programs. They are:

1. Interactive Approach
2. Batch mode Approach

**Program** → A set of optimized instructions is called a program.

**Optimized instructions** are nothing but instructions which take less memory space (space complexity) and less execution time (time complexity).

The purpose of developing or writing the program is that **“To develop Real Time problems or tasks”**.

After writing a set of optimized instructions, we save them on a computer with some file name with extension `.py` (source code).

#### 1. Interactive Approach

In Interactive Approach, a Python programmer can issue one statement at a time and gets one output at a time.

- Interactive Approach is most useful to test one instruction at a time.
- It is **not recommended** to write a group of instructions because we are unable to save these instructions and reuse them again over in another program/project.

**Example of Interactive Approach software:** 1. Python command prompt 2. Python IDLE Shell

Once we install Python software, we get the above two execution environments.

#### 2. Batch Mode Approach

In Batch Mode Approach, a Python programmer will write a group of instructions and save them into some file with the extension `.py` (source code) either for reuse or incorporating the file into another program/project.

Batch mode Approach is most useful to develop Real time applications in the form of programs.

**Example of Batch mode Approach software:** 1. Python IDLE (write mode) 2. Third party IDE (Integrated Development Environment) such as: - a) PyCharm - b) Anaconda / Jupiter Notebook - c) Anaconda / Spyder - d) Atom - e) VS Code - f) Google Colab

## Displaying Output of a Python Program

**Note:** To run the python program from Windows command prompt, we use a tool called “python”.

### Syntax:

```
python filename.py
```

To display the result of a Python program on the console, we use a pre-defined function called **print()**.

print() is used for displaying the result of a Python program on the console.

print() can be used in 6 ways. They are explained below.

### Syntax 1: print(value)

This syntax displays either a single value or multiple values.

```
print(value1, value2, ..., valueN)
```

```
a = 10
b = 20
c = a + b
print(a, b, c)
```

### Output:

```
10 20 30
```

### Syntax 2: print(message)

```
print(message)
```

or

```
print(message1, message2, ..., messageN)
```

or

```
print(message1 + message2 + message3 + ... + messageN)
```

This syntax displays message(s) which are of type str.

```
print("Hello world")
print('Hello world')
print("""Hello world""")
print('''Hello world''')
print('Hello', 'World')
print('Hello' + ' ' + 'World')
```

### Output:

```
Hello world
Hello world
Hello world
Hello world
```

```
Hello World
Hello World
```

When we try to concatenate a string with a non-string (int) value using +, Python raises a `TypeError`, since + can only concatenate str (not int) to str. We must convert the non-string value into str using `str()` first.

```
H = 4
# print("Hello" + H)    -> This raises: TypeError: can only concatenate str (not "int") to str
print("Hello" + str(H))
print("Hello" + str(4 * 3))
```

**Output:**

```
Hello4
Hello12
```

**Syntax 3: print(value cum message)**

```
print(value cum message)
```

or

```
print(message cum value)
```

This syntax displays values cum message or message cum values.

```
a = 10
print("value of =", a)
print(a, "is the value of a")

a = 10
b = 20
c = a + b
print("sum =", c)
print(c, "is the sum")
print("sum of", a, "and", b, "=", c)
print("sum of " + str(a) + " and " + str(b) + " = " + str(c))
print("sum of", (a, b), "=", c)
```

**Output:**

```
value of = 10
10 is the value of a
sum = 30
30 is the sum
sum of 10 and 20 = 30
sum of 10 and 20 = 30
sum of (10, 20) = 30
```

**Syntax 4: print(value/message with format())**

```
print("... {} ...".format(value))
```

or

```
print("{} ...".format(value))
```

This syntax displays value cum message or message cum value using the `format()` method with {} placeholders.

```

a = 10
print("value of a = {}".format(a))
print("{} is value of a.".format(a))

a = 10
b = 20
c = a + b
print("sum = {}".format(c))
print("{} is the sum".format(c))
print("sum of {} and {} = {}".format(a, b, c))
print("sum({}, {}) = {}".format(a, b, c))
print("{} + {} = {}".format(a, b, c))

sno = 10
sname = "Reyan"
print("my number is {} and Name is {}".format(sno, sname))

```

### Output:

```

value of a = 10
10 is value of a.
sum = 30
30 is the sum
sum of 10 and 20 = 30
sum(10, 20) = 30
10 + 20 = 30
my number is 10 and Name is Reyan

```

### Syntax 5: print(value/message with format specifiers)

```

print("... %<specifier> ..." % (value))
or
print("... %<specifier> ..." % (value1, value2, ...))

```

This syntax displays value cum message or message cum values **with format specifiers**.

In Python, we have 3 types of format specifiers. They are:

- a) %d → for Integer
- b) %f → for Float
- c) %s → for str

**Note:** When we come across mixed/advanced (heterogeneous) data, we must convert such type of data into str by using str() and use %s as the format specifier.

```

sno = 10
sname = "AB"
marks = 34.56
print("my number is %d and name is %s and marks is %0.2f" % (sno, sname, marks))

```

```

lst = [10, "brochure", 34.5, True]
print("content of lst = {}".format(lst))
print("content of lst = %s" % str(lst))
# print("content of lst = %d" % lst)    -> TypeError: %d format: a real number is required, not

```

```

d1 = {10: "Apple", 20: "mango", 30: "kiwi"}

```

```
print("content of d1 = {}".format(d1))
print("content of d1 = %s" % str(d1))
```

**Output:**

```
my number is 10 and name is AB and marks is 34.56
content of lst = [10, 'brochure', 34.5, True]
content of lst = [10, 'brochure', 34.5, True]
content of d1 = {10: 'Apple', 20: 'mango', 30: 'kiwi'}
content of d1 = {10: 'Apple', 20: 'mango', 30: 'kiwi'}
```

**Syntax 6: print(value, end="delimiter")**

```
print(value, end="delimiter")
```

This syntax displays value(s) in the same line — Horizontally.

**Ex:**

```
for x in range(1, 11, 1):
    print(x, end=' ')
```

This function (range(1, 11, 1)) is used from beginning to end for placing the values in row-wise (columns), displaying this message with a horizontal-arranged prompting message.

```
for x in range(1, 11, 1):
    print(x, end=' ')
print()
```

**Output:**

```
1 2 3 4 5 6 7 8 9 10
```

## Reading the Data Dynamically from Keyboard

To read the data Dynamically from keyboard, we use two pre-defined functions. They are:

- a) input()
- b) input(message)

**1. input()**

**Syntax:** varname = input()

This function is used for reading any type of data from keyboard and placed in varname in the form of <class 'str'> (str content).

Programmatically, we can convert str type data into other type of values by using **Type Casting functions**.

**2. input(message)**

**Syntax:** varname = input(message)

This function is used for reading any type of data from keyboard and placed in varname in the form of <class 'str'>. Additionally, it gives a user-prompting message.

Here message represents the user-prompting message.

Programmatically, we can convert str type data into other type of values by using Type Casting functions.

## Programming Questions

The following programs are based on the questions listed at the end of the notes.

### Q1. WAP to find area and perimeter of a square

```
side = float(input("Enter side of the square: ") or "5")
area = side ** 2
perimeter = 4 * side
print("Area of square = {}".format(area))
print("Perimeter of square = {}".format(perimeter))

# Demonstration run with a sample value (side = 5)
side = 5
area = side ** 2
perimeter = 4 * side
print("Area of square = {}".format(area))
print("Perimeter of square = {}".format(perimeter))
```

#### Output:

```
Area of square = 25
Perimeter of square = 20
```

### Q2. WAP to find area of a circle

```
radius = float(input("Enter radius of the circle: ") or "7")
pi = 3.14159
area = pi * radius ** 2
print("Area of circle = {}".format(area))

radius = 7
pi = 3.14159
area = pi * radius ** 2
print("Area of circle = {}".format(area))
```

#### Output:

```
Area of circle = 153.93791
```

### Q3. WAP which demonstrates the functionality of arithmetic operations

```
a = 10
b = 3
print("a + b =", a + b)
print("a - b =", a - b)
print("a * b =", a * b)
print("a / b =", a / b)
print("a // b =", a // b)
print("a % b =", a % b)
print("a ** b =", a ** b)
```

#### Output:

```
a + b = 13
a - b = 7
a * b = 30
a / b = 3.3333333333333335
```

```
a // b = 3
a % b = 1
a ** b = 1000
```

#### Q4. WAP to calculate the square root of any number without using sqrt() function of the math module

**Note:** In Python,  $n ** 0.5$  is the same as  $n ** (1/2)$ , which gives the square root of  $n$ .

```
n = int(input("Enter no. for getting its square root: ") or "25")
res = n ** 0.5
print("sqrt({}) = {}".format(n, res))
print("sqrt(%d) = %0.2f" % (n, res))
print("sqrt({}) = {}".format(n, round(res, 2)))

n = 25
res = n ** 0.5
print("sqrt({}) = {}".format(n, res))
print("sqrt(%d) = %0.2f" % (n, res))
print("sqrt({}) = {}".format(n, round(res, 2)))
```

#### Output:

```
sqrt(25) = 5.0
sqrt(25) = 5.00
sqrt(25) = 5.0
```

#### Q5. WAP for $(a+b)**2$

```
(a+b)**2 = (a+b) * (a+b)
          = a*a + 2*a*b + b*b
          = a**2 + 2*a*b + b**2

a = 10
b = 20
result1 = (a + b) ** 2
result2 = (a + b) * (a + b)
result3 = a ** 2 + 2 * a * b + b ** 2
print("(a+b)**2 =", result1)
print("(a+b)*(a+b) =", result2)
print("a**2 + 2ab + b**2 =", result3)
```

#### Output:

```
(a+b)**2 = 900
(a+b)*(a+b) = 900
a**2 + 2ab + b**2 = 900
```

#### Q6. WAP to calculate $a**m / a**n$

```
a**m / a**n = a**(m-n)
(a**m)**n = a**(m*n)

a = 2
m = 5
n = 2
direct = (a ** m) / (a ** n)
using_law = a ** (m - n)
```

```

power_law = (a ** m) ** n
combined_power = a ** (m * n)
print("a**m / a**n =", direct)
print("a**(m-n) =", using_law)
print("(a**m)**n =", power_law)
print("a**(m*n) =", combined_power)

```

**Output:**

```

a**m / a**n = 8.0
a**(m-n) = 8
(a**m)**n = 1024
a**(m*n) = 1024

```

**Q7. WAP which accepts number of days and converts into number of years and months**

```

days = int(input("Enter no. of days: ") or "800")
years = days // 365
remaining_days = days % 365
months = remaining_days // 30
final_days = remaining_days % 30
print("{} days = {} years, {} months, {} days".format(days, years, months, final_days))

days = 800
years = days // 365
remaining_days = days % 365
months = remaining_days // 30
final_days = remaining_days % 30
print("{} days = {} years, {} months, {} days".format(days, years, months, final_days))

```

**Output:**

```

800 days = 2 years, 2 months, 10 days

```

**Q8. WAP which computes the value of an arithmetic expression using multi-line assignment**

```

a, b, c = 5, 3, 2
result = a * b + a - b * c
print("a, b, c =", a, b, c)
print("a*b + a - b*c =", result)

```

**Output:**

```

a, b, c = 5 3 2
a*b + a - b*c = 14

```

**Q9. WAP which computes all types of Relational and Assignment operators**

```

a = 10
b = 20

# Relational operators
print("Relational Operators:")
print("a > b :", a > b)
print("a < b :", a < b)

```



```

print("a >= b :", a >= b)
print("a <= b :", a <= b)
print("a == b :", a == b)
print("a != b :", a != b)

# Assignment operators
print()
print("Assignment Operators:")
c = a
print("c = a ->", c)
c += a
print("c += a ->", c)
c -= a
print("c -= a ->", c)
c *= a
print("c *= a ->", c)
c /= a
print("c /= a ->", c)
c **= 2
print("c **= 2 ->", c)
c %= a
print("c %= a ->", c)

```

### **Output:**

Relational Operators:

```

a > b : False
a < b : True
a >= b : False
a <= b : True
a == b : False
a != b : True

```

Assignment Operators:

```

c = a -> 10
c += a -> 20
c -= a -> 10
c *= a -> 100
c /= a -> 10
c **= 2 -> 100
c %= a -> 0

```